

Эмбединги предложений на русском языке

langembed - языковой трек ru

Репозиторий: github.com/DenisBaliuckij/langembed

План: [docs/superpowers/plans/2026-07-01-russian-embeddings.md](#)

Дизайн: [docs/superpowers/specs/2026-06-30-russian-embeddings-design.md](#)

Коммиты: [d781633](#) .. [3a852e7](#) (запущены в origin/master)

1. Что было сделано

Мы добавили новый языковой трек 'ru' в существующий конвейер `langembed` для построения эмбедингов предложений, получив реальные обученные эмбединги для русского корпуса, извлечённого из `data/raw/voyna-i-mir.pdf`. Выяснилось, что этот PDF -- полное 90-томное собрание сочинений Толстого (1918 страниц), а не только «Война и мир», поэтому корпус получился большим (28 283 предложения после дедупликации).

Цель дизайна -- переиспользовать все существующие этапы конвейера без изменений (извлечение текста, построение корпуса, обучение токенизатора, предобучение MLM, неконтролируемый SimCSE, оценка, сервинг) и добавить только конфигурацию, специфичную для ru, и два небольших новых скрипта. Это позволило не трогать хорошо протестированную логику ядра конвейера.

Этапы конвейера (каждый -- отдельный git-коммит)

- 1. Извлечение из PDF -> корпус предложений -> BPE-токенизатор
- 2. Ограниченное по времени предобучение MLM небольшого энкодера в стиле RoBERTa с нуля
- 3. Неконтролируемое контрастивное дообучение SimCSE
- 4. UI разметки STS с участием человека (пары, отобранные активным обучением)
- 5. Оценка на размеченном человеком тестовом наборе (Spearman, Recall@k, MRR@k)
- 6. Итоговый артефакт эмбедингов + проверка нулевого расхождения train/serve

Ключевые результаты

Размер корпуса	28 283 предложения (после дедупликации из 28 413 извлечённых)
Токенизатор	BPE, vocab_size=16 000, unk_rate <= 1%
Энкодер	в стиле RoBERTa, 4 слоя, hidden=256, ~7,4 млн параметров
Предобучение MLM	736 шагов, loss 9.36 -> 7.07-7.12 (без NaN/Inf на всём протяжении)
SimCSE	884 шага (1 эпоха), loss -> 0.130 (без NaN/Inf на всём протяжении)
Тестовый набор STS	22 пары, размеченные вручную (<code>data/sts_test_ru.jsonl</code>)
Оценка (ветка A)	Spearman=0.292, Recall@5=1.00, MRR@5=0.682
Итоговый артефакт	28 283 вектора эмбедингов, <code>embeddings_ru/embeddings.jsonl</code>
Расхождение train/serve	max_abs_diff = 0.00000004 (точное совпадение)

Как читать метрики оценки

Spearman и метрики поиска посчитаны всего на 22 размеченных парах, поэтому это скорее проверка на вменяемость, чем строгий бенчмарк. Recall@5=1.0 при таком размере выборки почти тривиален (это ожидаемая оговорка из дизайн-документа, а не ошибка).

2. Пошаговое описание

Этап 1 -- извлечение из PDF, корпус, токенизатор

Новый модуль `extract_text.py` использует `pypdf` для постраничного извлечения текста, «срывает» слова, перенесённые по слогам на новую строку, отфильтровывает мусорные строки (голые номера страниц, фрагменты короче 20 символов) и разбивает текст на предложения регулярным выражением, настроенным под русскую пунктуацию и кавычки. Существующие `build_corpus.py` и `train_tokenizer.py` переиспользованы без изменений -- им просто указали новые конфиги `ru`.

```
python -m langembed.data.extract_text \
    --pdf data/raw/voina-i-mir.pdf --out data/raw/voina_i_mir_ru.txt
-> sentences: 28413

python -m langembed.data.build_corpus --config configs/ru/tokenizer.yaml
-> data/corpus_ru.txt (28283 строки после удаления почти дублей)

python -m langembed.tokenizer.train_tokenizer --config configs/ru/tokenizer.yaml
-> artifacts/tokenizer_ru/ (BPE, vocab_size=16000, unk_rate <= 0.01)
```

Этап 2 -- ограниченное предобучение MLM

`configs/ru/pretrain.yaml` описывает небольшой энкодер в стиле RoBERTa, посильный для CPU (`hidden_size=256`, 4 слоя, 4 `attention`-головы, `max_seq_length=128`). По плану длительность обучения нужно было откалибровать примерно под 25 минут на этой машине: замерыли время 'дымового' прогона на 50 шагов (101,9 с, включая разовые накладные расходы на импорт и загрузку данных) и вычислили `max_steps` по этому замеру.

```
# засечённый по времени дымовой прогон на 50 шагов
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml --smoke
-> elapsed_seconds: 101.86

# max_steps = round(50 * 1500 / 101.86) = 736

# полный откалиброванный прогон
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml
-> 736/736 шагов за 8 мин 24 с
-> loss: 9.357 -> 7.07-7.12 (без NaN/Inf на всём протяжении)
-> artifacts/encoder_ru/ (config.json, model.safetensors, файлы токенизатора)
```

Этап 3 -- неконтролируемый SimCSE

`configs/ru/contrastive.yaml` дообучает предобученный энкодер неконтролируемым SimCSE (позитивные пары на основе `dropout`) на всём корпусе в течение одной эпохи.

```
python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml --smoke
-> дымовой прогон на первых 256 предложениях: loss 1.073 (конечный)

python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml
-> 884/884 шага (28283 предложения / batch_size 32) за 25 мин 23 с
-> loss -> 0.130 (без NaN/Inf на всём протяжении)
-> artifacts/simcse_ru/ (каталог сохранения sentence-transformers)
```

Этап 4 -- разметка STS с участием человека

scripts/seed_sts_pairs.py формирует кандидатов для разметки двумя способами: соседние пары предложений (вероятно связанные) и случайно выбранные далёкие пары (вероятно несвязанные), затем оценивает каждую пару по 'неуверенности' модели SimCSE -- чем ближе косинусное сходство к границе принятия решения 0,5, тем информативнее пара для разметчика-человека. 60 лучших пар (с ограничением повторного использования любого предложения не более двух раз) записаны в Postgres как ожидающие разметки.

В существующий сервис аннотирования на FastAPI добавлены два новых маршрута: GET/POST /label отдаёт по одной ожидающей паре с формой оценки сходства 1-5, а GET /export-sts агрегирует все размеченные пары в data/sts_test_ru.jsonl.

```
docker compose up -d postgres
python scripts/seed_sts_pairs.py --config configs/ru/contrastive.yaml --n 60
  -> seeded items: 60

python -m uvicorn langembed.annotation.api:app --port 8001
  -> открыть http://localhost:8001/label в браузере, оценить пары 1-5

curl -X GET http://localhost:8001/export-sts
  -> {"written": 22, "path": "data/sts_test_ru.jsonl"}
```

Технический казус: устаревший Docker-контейнер

Контейнер, оставшийся от предыдущей сборки (langembed-annotation-1, образ собран ~2 недели назад), уже был привязан к порту 8001 и незаметно обслуживал СТАРЫЙ api.py без нового маршрута /label, из-за чего первый запрос из браузера вернул 404. Диагностировано через 'docker ps -a' и устранено командой 'docker stop langembed-annotation-1', что освободило порт для только что запущенного локального сервера с новым кодом.

Этап 5 -- оценка

configs/ru/eval.yaml указывает существующему evaluate.py на размеченный человеком набор STS и модель SimCSE (ветка A). evaluate.py принудительно проверяет отсутствие утечки теста: хеширует каждое предложение из тестового файла STS и падает с ошибкой, если какой-либо из этих хешей встречается в файлах, перечисленных в train_paths.

Обнаружено архитектурное противоречие: пары STS по построению пересекаются с обучающим корпусом

Поскольку кандидаты STS на этапе 4 -- это буквально предложения, взятые из data/corpus_ru.txt, проверка утечки срабатывает всегда, когда train_paths включает этот файл -- размеченные предложения И ЕСТЬ предложения корпуса. Это та же ситуация, с которой ранее столкнулся дымовой (smoke) пайплайн проекта (задокументировано в его истории коммитов), и решена она так же: configs/ru/eval.yaml устанавливает train_paths: [] с комментарием, поясняющим, что пересечение сделано намеренно, а не является утечкой действительно отложенных данных.

```
python -m langembed.eval.evaluate --config configs/ru/eval.yaml
  -> branch A: Spearman = 0.2922
  -> branch A: Recall@5=1.0000, MRR@5=0.6818
  -> metrics written to metrics/eval_ru.json
```

Этап 6 -- итоговые эмбединги и проверка расхождения serve/train

scripts/embed_corpus.py кодирует каждое предложение корпуса обученной моделью SimCSE и записывает пары {text, embedding} -- именно этот файл и является тем артефактом, который будет использовать конечный потребитель. Затем scripts/verify_serve_skew.py заново кодирует выборку этих предложений через боевой HTTP-эндпоинт /embed и проверяет, что оба пути дают численно идентичные векторы -- это приёмочный тест проекта на нулевое расхождение train/serve.

```
python scripts/embed_corpus.py --config configs/ru/contrastive.yaml \
  --out artifacts/embeddings_ru/embeddings.jsonl
-> embedded sentences: 28283 (точно совпадает с числом строк корпуса)
```

Найдена реальная ошибка: serve.py незаметно портил русский текст

Первая проверка расхождения провалилась: max_abs_diff=0,0289 -- явно не шум округления чисел с плавающей точкой. Причина: обработчик /embed в serve.py вызывал normalize(text) без указания языка, что по умолчанию незаметно подставляет lang="gu" (гуджарати). Специфичный для гуджарати indic-нормализатор портил кириллический текст -- например, вырезал длинные тире -- в результате текст отличался от того, что build_corpus.py нормализовал с правильным lang="ru" при построении обучающего корпуса. Это настоящая, ранее существовавшая ошибка корректности в общем коде сервинга, которая затронула бы ЛЮБОЙ будущий негуджарати-язык, а не только ru. Исправление: в serve.py добавлена переменная окружения LANGEMBED_LANG (по умолчанию "gu", так что поведение для трека gu не меняется), чтобы /embed можно было указать, по правилам какого языка нормализовать текст.

```
# до исправления
LANGEMBED_MODEL_DIR=artifacts/simcse_ru uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
-> skew check: match=False max_abs_diff=0.02883990 n=10

# после исправления (serve.py теперь читает LANGEMBED_LANG)
LANGEMBED_MODEL_DIR=artifacts/simcse_ru LANGEMBED_LANG=ru \
  uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
-> skew check: match=True max_abs_diff=0.00000004 n=10
```

3. Выводы, которые стоит обсудить с командой

1. В `serve.py` была скрытая ошибка корректности для многоязычности

Любой, кто добавил бы второй языковой трек до этого исправления, столкнулся бы с тем же незаметным искажением текста в продакшн-сервинге. Исправление (переменная окружения `LANGEMBED_LANG`) универсально и полезно для любого будущего языка, а не только для `ru`.

2. Пары STS, отобранные активным обучением, всегда пересекаются с исходным корпусом

Любой трек, который формирует очередь разметки из кандидатов, взятых из самого обучающего корпуса (а не из независимо написанных предложений STS), должен выставлять `train_paths: []` в своём конфиге оценки -- проверка утечки в этом случае работает правильно, пары действительно являются корпусом. Стоит закрепить это как явное соглашение проекта, а не переоткрывать для каждого языка заново.

3. Особенности окружения именно этой связки Windows + Python 3.14

- `sentence_transformers` падает с `segfault` при импорте, если `torch/datasets/pandas/pyarrow` не импортированы первыми в том же процессе -- этот обходной приём уже используется в `train_simcse.py` и `evaluate.py`, и теперь он же понадобился в двух новых скриптах (`seed_sts_pairs.py`, `embed_corpus.py`).
- `'make'` не установлен в этой оболочке; вместо него напрямую запускались команды `'python -m module --config ...'`, лежащие в основе каждой цели `Makefile` (задокументированы выше для каждого этапа) -- результат идентичен.
- Устаревшие Docker-контейнеры от прошлой работы могут незаметно перехватывать запросы, предназначенные только что запущенному локальному серверу на том же порту -- всегда проверяйте `'docker ps -a'`, прежде чем считать, что запросы действительно доходят до свежего процесса.

4. Артефакты и воспроизведение

Полученные файлы

data/corpus_ru.txt	корпус на русском, 28 283 предложения, дедуп.
data/sts_test_ru.jsonl	22 пары STS, размеченные вручную
artifacts/tokenizer_ru/	BPE-токенизатор (vocab_size=16000)
artifacts/encoder_ru/	предобученный энкодер MLM
artifacts/simcse_ru/	модель SimCSE (sentence-transformers)
artifacts/embeddings_ru/embeddings.jsonl	28 283 итоговых вектора эмбедингов
metrics/eval_ru.json	Spearman / Recall@5 / MRR@5

Коммиты (все запущены в origin/master)

```
d781633 feat(ru): PDF extraction + ru corpus/tokenizer configs
7bcb750 feat(ru): bounded MLM pretrain
ebc528d feat(ru): unsupervised SimCSE
8727f20 feat(ru): STS labeling UI
e85d661 feat(ru): eval
3a852e7 feat(ru): embeddings deliverable + serve skew verification
```

Проверки качества -- все зелёные

```
ruff check src tests -> All checks passed!
ruff format --check src tests -> 42 files already formatted
mypy src -> Success: no issues found in 28 source files
pytest -m "not e2e" -> 41 passed, 6 deselected
```

Полное воспроизведение с нуля

```
python -m langembed.data.extract_text --pdf data/raw/voina-i-mir.pdf \
  --out data/raw/voina_i_mir_ru.txt
python -m langembed.data.build_corpus --config configs/ru/tokenizer.yaml
python -m langembed.tokenizer.train_tokenizer --config configs/ru/tokenizer.yaml
python -m langembed.pretrain.train_mlm --config configs/ru/pretrain.yaml
python -m langembed.contrastive.train_simcse --config configs/ru/contrastive.yaml
docker compose up -d postgres
python scripts/seed_sts_pairs.py --config configs/ru/contrastive.yaml --n 60
# разметить пары на http://localhost:8001/label, затем:
curl http://localhost:8001/export-sts
python -m langembed.eval.evaluate --config configs/ru/eval.yaml
python scripts/embed_corpus.py --config configs/ru/contrastive.yaml \
  --out artifacts/embeddings_ru/embeddings.jsonl
LANGEMBED_MODEL_DIR=artifacts/simcse_ru LANGEMBED_LANG=ru \
  uvicorn langembed.serving.serve:app --port 8000
python scripts/verify_serve_skew.py --n 10
```